

AI ASSISTED CODING

ASSIGNMENT-17.1

Name: Guda Harshitha

H.T.No:2303A51102

Task 1: Perform EDA on Penguin dataset(from seaborn)

- Load and display entire dataset
- Display country wise count
- Visualize bar chart (country wise)
- Visualize bar chart with values count at top
- Visualize pairplot and other plots

```
[1] ✓ In
import seaborn as sns
import matplotlib.pyplot as plt

# Load the Penguin dataset
penguins_df = sns.load_dataset('penguins')
```

▼ Displaying the entire dataset

```
[2] ✓ In
# Display the first 5 rows of the dataset
display(penguins_df.head())

# Display general information about the dataset
print(penguins_df.info())

# Display descriptive statistics of the dataset
display(penguins_df.describe())
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 7 columns):
 #   Column                Non-Null Count  Dtype  
---  --
 0   species                344 non-null   object  
 1   island                 344 non-null   object  
 2   bill_length_mm         342 non-null   float64  
 3   bill_depth_mm          342 non-null   float64  
 4   flipper_length_mm      342 non-null   float64
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	342 000000	342 000000	342 000000	342 000000
mean	43.921530	17.151170	200.915205	4201.754386
std	5.459504	1.974793	14.061714	801.954536
min	32.100000	13.100000	172.000000	2700.000000
25%	39.225000	15.600000	196.000000	3550.000000
50%	44.450000	17.300000	197.000000	4050.000000
75%	48.500000	18.700000	213.000000	4750.000000
max	59.600000	21.500000	231.000000	6300.000000

Displaying species-wise count

Since the dataset doesn't have a 'country' column, I'll display the count of observations per 'species', which is a common categorical variable for analysis in this dataset. If you had a different column in mind, please let me know!

```
# Display the count of each species
species_counts = penguins_df['species'].value_counts()
display(species_counts)
```

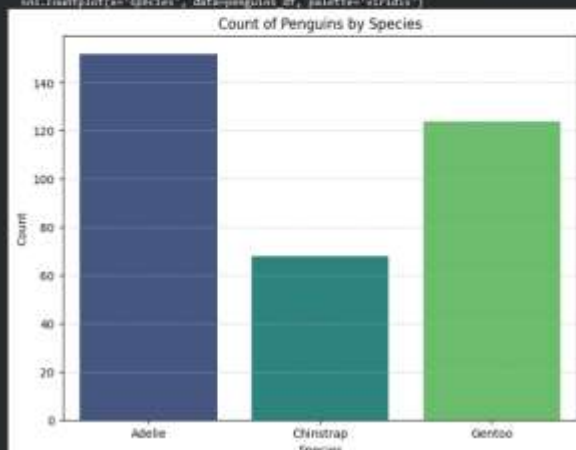
```
count
species
Adelie    152
Gentoo    124
Chinstrap  68
dtype: int64
```

```
plt.figure(figsize=(8, 8))
sns.countplot(x='species', data=penguins_df, palette='viridis')
plt.title('Count of Penguins by Species')
plt.xlabel('Species')
plt.ylabel('Count')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

/tmp/ipykernel_3776/3462765654.py:12: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.countplot(x='species', data=penguins_df, palette='viridis')
```



Visualizing bar chart with values count at top

```
plt.figure(figsize=(8, 6))
ax = sns.countplot(x='species', data=penguins_df, palette='viridis')
plt.title('Count of Penguins by Species with Values')
plt.xlabel('species')
plt.ylabel('Count')
plt.grid(True, linestyle='--', alpha=0.7)

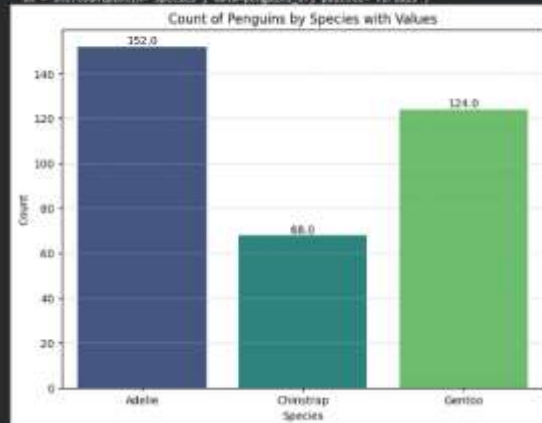
# Add the count values on top of the bars
for p in ax.patches:
    ax.annotate(p.get_height(), (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center', fontsize=10, color='black', xytext=(0, 1),
                textcoords='offset points')

plt.show()
```

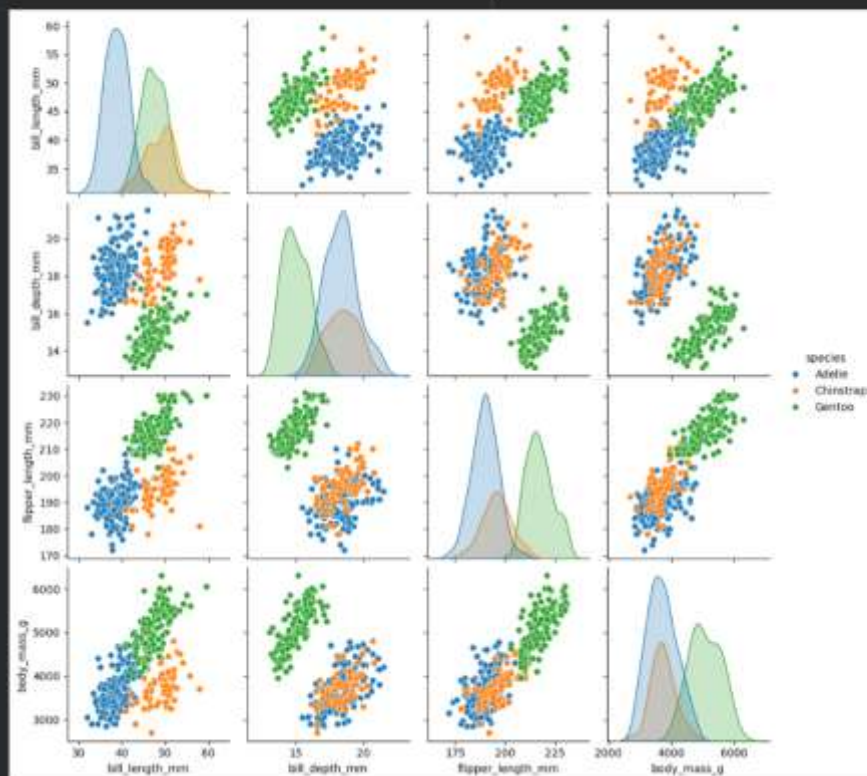
/tmp/ipykernel_3779/339490046.py:11: FutureWarning:

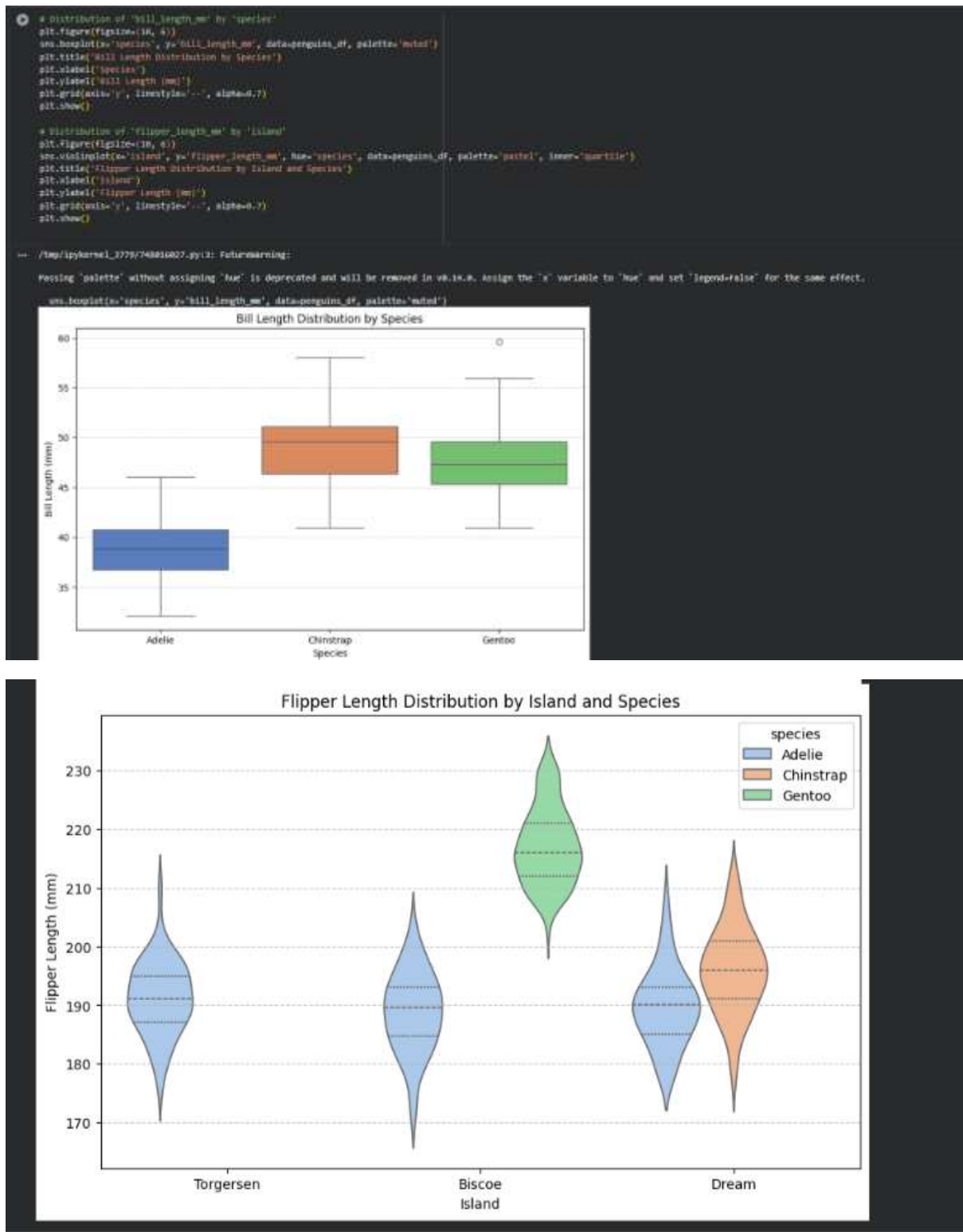
Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
ax = sns.countplot(x='species', data=penguins_df, palette='viridis')
```



```
# Pairplot to visualize relationships between numerical variables
sns.pairplot(penguins_df, hue='species', diag_kind='hist')
plt.title('Pairplot of Penguin Dataset by Species', y=1.40) # Adjust title position
plt.show()
```





Task 2: Perform EDA on HealthExp dataset(seaborn)

- Load and display entire dataset
- Display columns and statistical summary(describe)
- Display missing values , and draw bar chart

- Visualize pairplot, regplot and box plot, scatter plot etc

```
[8]
✓ 3s
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Load the HealthExp dataset
health_exp_df = sns.load_dataset('healthexp')
```

Load and display entire dataset

```
[9]
✓ 0s
# Display the first 5 rows of the dataset
display(health_exp_df.head())
```

	Year	Country	Spending_USD	Life_Expectancy
0	1970	Germany	252.311	70.6
1	1970	France	192.143	72.2
2	1970	Great Britain	123.993	71.9
3	1970	Japan	150.437	72.0
4	1970	USA	326.961	70.9

Display columns and statistical summary

```
[10]
✓ 0s
# Display general information about the dataset
print(health_exp_df.info())

# Display descriptive statistics of the dataset
display(health_exp_df.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 274 entries, 0 to 273
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Year            274 non-null   int64
1   Country         274 non-null   object
2   Spending_USD    274 non-null   float64
3   Life_Expectancy 274 non-null   float64
dtypes: float64(2), int64(1), object(1)
memory usage: 8.7+ KB
None
```

	Year	Spending_USD	Life_Expectancy
count	274.000000	274.000000	274.000000
mean	1996.992701	2789.338905	77.909489
std	14.180933	2194.939785	3.276263
min	1970.000000	123.993000	70.600000
25%	1985.250000	1038.357000	75.525000
50%	1998.000000	2295.578000	78.100000
75%	2009.000000	4055.610000	80.575000
max	2020.000000	11859.179000	84.700000

Display missing values and draw bar chart

```
# Display missing values count
missing_values = health_exp_df.isnull().sum()
display(missing_values[missing_values > 0])

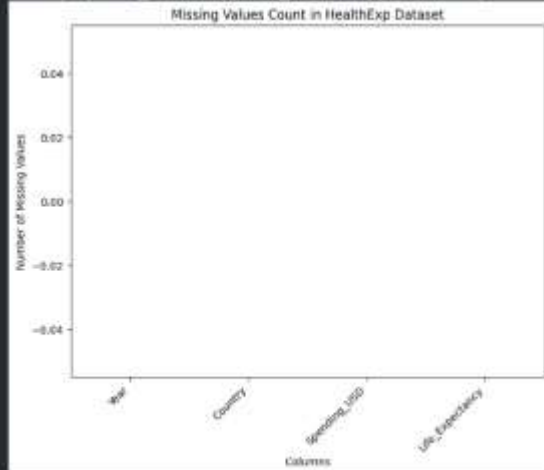
# Visualize missing values
plt.figure(figsize=(8, 4))
sns.barplot(missing_values.index, y=missing_values.values, palette='viridis')
plt.title('Missing Values Count in HealthExp Dataset')
plt.xlabel('Columns')
plt.ylabel('Number of Missing values')
plt.xticks(rotation=45, loc='right')
plt.show()
```

Output:

/tmp/ipykernel_1776/1899401395.py:7: FutureWarning:

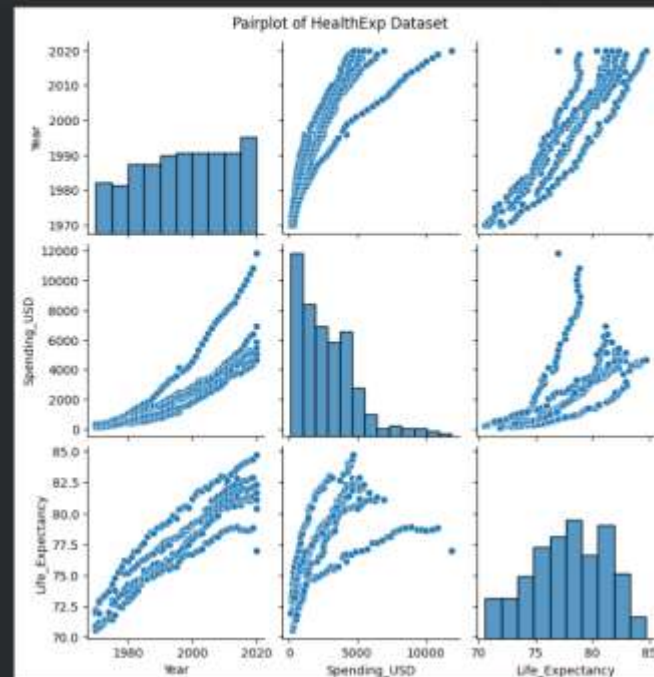
Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

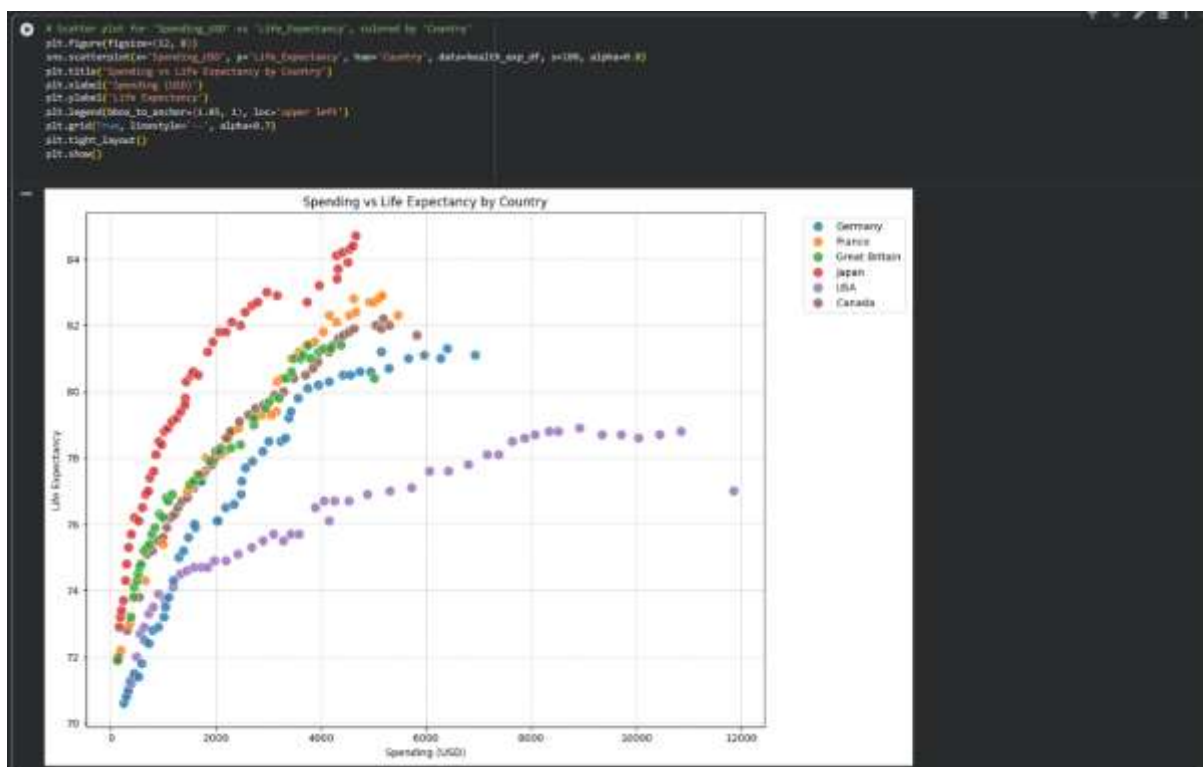
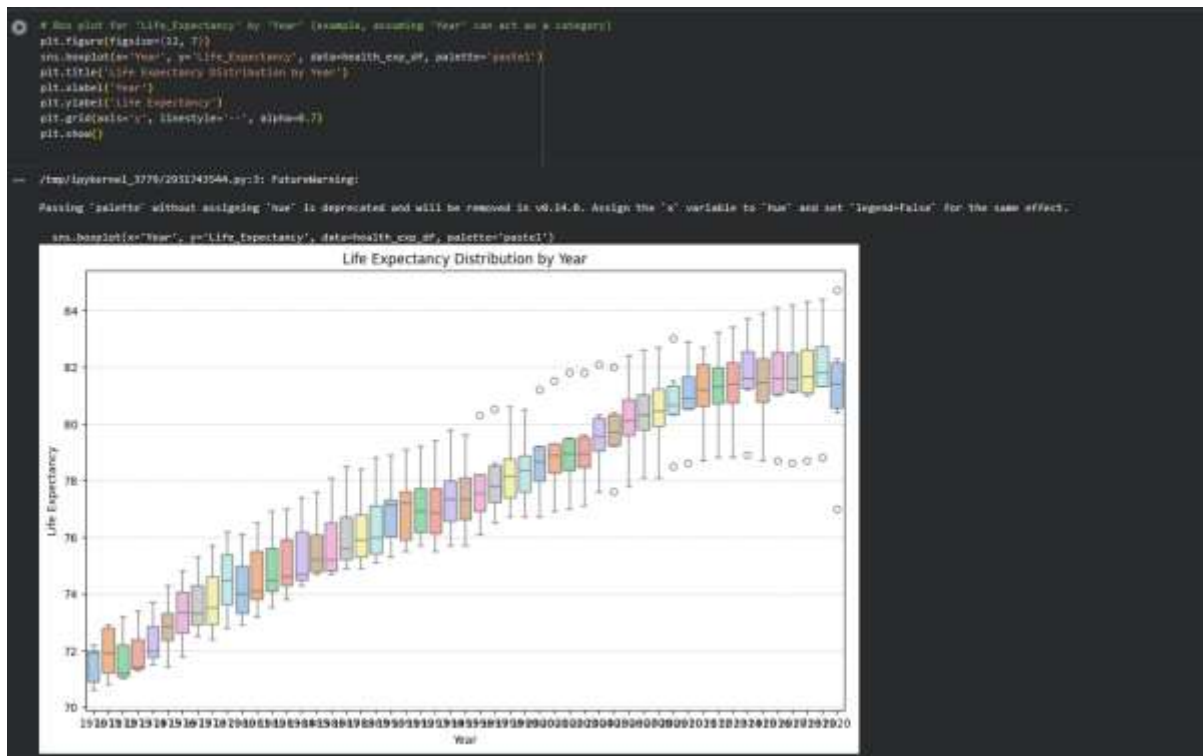
See [https://matplotlib.org/3.1.1/whatsnew/2.0.html#palette-without-hue-argument](#) for more information.



Visualize pairplot, regplot and box plot, scatter plot

```
# Pairplot to visualize relationships between numerical variables
sns.pairplot(health_exp_df)
plt.suptitle('Pairplot of HealthExp Dataset', y=1.02) # Adjust title position
plt.show()
```





Task 3: Perform following task on given dataset (Train the model)

- Analyze the Planets dataset to understand its structure, features, and missing values.
- Perform data cleaning and preprocessing to handle missing or incorrect data.
- Train the dataset using different machine learning models to make predictions.

- Compare and explain the performance of all trained models with proper reasoning.

```

100 import pandas as pd
101 import numpy as np
102 import matplotlib.pyplot as plt
103 import seaborn as sns

104 # Load the Planets dataset
105 planets_df = sns.load_dataset('planets')

106
107 - Analyze the Planets dataset: Structure, Features, and Missing Values
108
109 # Display the first 5 rows to understand the data structure
110 display(planets_df.head())

111 # Display general information about the dataset (columns, non-null counts, dtypes)
112 print(planets_df.info())

113 # Display descriptive statistics for numerical columns
114 display(planets_df.describe())

```

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	384.235	7.16	77.45	2004
1	Radial Velocity	1	814.774	8.21	30.06	2004
2	Radial Velocity	1	782.905	2.09	49.24	2011
3	Radial Velocity	1	525.930	70.49	110.52	2007
4	Radial Velocity	1	616.236	10.38	110.47	2004

```

115 # Create 'planet_name' from 'method' and 'number'
116 planets_df['planet_name'] = planets_df['method'] + ' ' + planets_df['number']
117 # Drop 'method' and 'number' columns
118 planets_df.drop(['method', 'number'], axis=1, inplace=True)
119 # Display the updated dataset
120 display(planets_df.head())
121
122 # Display general information about the dataset (columns, non-null counts, dtypes)
123 print(planets_df.info())
124
125 # Display descriptive statistics for numerical columns
126 display(planets_df.describe())
127
128 # Types: float64(5), int64(2), object(5)
129 memory usage: 28.4+ KB
130
131
132      number  orbital_period      mass  distance      year
count  1530.000000      961.000000  913.000000  806.000000  1530.000000
mean     1.766667      2802.677666  2.024181  384.064152  2006.675033
std      1.248078      28014.720324  5.018817   723.178483   5.672987
min       1.000000       6.000700   0.060000   1.300000  1998.000000
25%       1.000000      2.442040   0.224000  32.590000  2007.500000
50%       1.000000     28.679000   1.280000  52.320000  2010.000000
75%       1.000000     608.024000  1.040000  115.000000  2013.000000
max       7.000000  720000.000000  25.000000  9600.000000  2014.000000

```

```

131
132 # Calculate and display the percentage of missing values for each column
133 missing_percentage = planets_df.isnull().sum() * 100 / len(planets_df)
134 missing_data = pd.DataFrame({'Missing_Percentage': missing_percentage, 'Missing_Percentage': missing_percentage * 0.1})
135 display(missing_data)
136
137 # Visualize missing values
138 if not missing_data.empty:
139     plt.figure(figsize=(10, 8))
140     sns.barplot(missing_data.index, missing_data['Missing_Percentage'], palette='magma')
141     plt.title('Percentage of Missing Values per Column')
142     plt.xlabel('Column Name')
143     plt.ylabel('Missing Percentage (%)')
144     plt.xticks(rotation=45, label='right')
145     plt.tight_layout()
146     plt.show()
147
148 # Print the missing values found in the dataset
149 print("Missing values found in the dataset:")

```

Missing Percentage
mass
distance
orbital_period

Handling 'planet_name' without assigning 'True' is deprecated and will be removed in v0.24.0. Assign the 'x' variable to 'True' and set 'legend=False' for the same effect.

```

149 # Create a bar chart showing the percentage of missing values per column
150 sns.barplot(x=missing_data.index, y=missing_data['Missing_Percentage'], palette='magma')

```

Column Name	Missing Percentage (%)
mass	~50
distance	~22
orbital_period	~5

Based on the missing values analysis, we will address the NaNs. The `orbital_period` column has a significant number of missing values. Since this is a crucial feature, and we aim to predict it, we will drop rows where `orbital_period` is missing. For other numerical columns like `mass` and `distance`, we will impute missing values with the mean. The `method` column is categorical and will be one-hot encoded. The `year` column is important for time-series context, and `number` represents the number of planets found in the system, which can be useful.

We will define `orbital_period` as our target variable (y) and other processed features as our input (X) for a regression task.

```

# Drop rows where the target variable 'orbital_period' is missing
planets_df_cleaned = planets_df.dropna(subset=['orbital_period']).copy()

# Impute missing values in 'mass' and 'distance' with their respective means
# Using .fillna() on a copy of the DataFrame
planets_df_cleaned['mass'].fillna(planets_df_cleaned['mass'].mean(), inplace=True)
planets_df_cleaned['distance'].fillna(planets_df_cleaned['distance'].mean(), inplace=True)

# Handle categorical features: 'method' column using one-hot encoding
planets_df_processed = pd.get_dummies(planets_df_cleaned, columns=['method'], drop_first=True, dtype=int)

# Display the info of the processed DataFrame to check for remaining NaNs and new columns
print("\nInfo after cleaning and preprocessing:")
print(planets_df_processed.info())

# Display the head of the processed DataFrame
print("\nHead after cleaning and preprocessing:")
display(planets_df_processed.head())

```

[illegible]

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

X_train shape: (793, 13)
X_test shape: (199, 13)
y_train shape: (793,)
y_test shape: (199,)

```

Model 1: Linear Regression

```

# Initialize and train the linear Regression model
lin_reg_model = LinearRegression()
lin_reg_model.fit(X_train, y_train)

# Make predictions
y_pred_lr = lin_reg_model.predict(X_test)

# Evaluate the model
mse_lr = mean_squared_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print(f"Linear Regression - Mean Squared Error: {mse_lr:.2f}")
print(f"Linear Regression - R-squared: {r2_lr:.2f}")

Linear Regression - Mean Squared Error: 2768473063.11
Linear Regression - R-squared: 0.32

```

Task 4: Perform following task on given dataset (Train the model)

- Load the MNIST dataset from PyTorch.
- Display basic information and visualize sample input images with their class labels.
- Perform data preprocessing
- Design a EfficientNet model for imageclassification.
- Train the CNN model (10 epochs)
- Evaluate model performance using accuracy and loss metrics.
- Visualize the training history (accuracy and loss curves).

```

101 # Export necessary libraries
102 import torch
103 import torchvision
104 import torchvision.transforms as transforms
105 import matplotlib.pyplot as plt
106 import numpy as np

# Define a transformation to convert images to tensors and normalize them
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

# Load the MNIST training and test datasets
train_dataset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=transform)

# Create data loaders
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=1000, shuffle=False)

print("Number of training samples: {}".format(len(train_dataset)))
print("Number of test samples: {}".format(len(test_dataset)))
print("Image shape: {}".format(train_dataset[0][0].shape))
print("Number of classes: {}".format(len(train_dataset.classes)))

Number of training samples: 60000
Number of test samples: 10000
Image shape: torch.Size([1, 28, 28])
Number of classes: 10

```



Perform Data Preprocessing

For the MNIST dataset, basic preprocessing usually involves converting images to tensors and normalization. This has already been applied in the `transform` object when loading the dataset. MNIST images are grayscale (1 channel), 28x28 pixels. The `transforms.ToTensor()` converts them to `[C, H, W]` format (1, 28, 28) and scales pixel values to [0, 1]. `transforms.Normalize()` then normalizes these values using the mean and standard deviation of the MNIST dataset.

No further complex preprocessing like resizing or augmentation is typically needed for a standard MNIST classification task with a simple CNN, but it might be considered for more advanced models or higher accuracy goals. Given the task description, the current preprocessing is sufficient.

Task 5 – Customer Data Cleaning

Task:

Use AI to generate a Python script for cleaning a customer dataset.

Instructions:

- Handle missing values in columns (age, income, city).
- Remove duplicate records.
- Standardize text values (e.g., city names in lowercase).
- Encode categorical variables (gender, city).

Sample Code:

```
import pandas as pd

data = pd.read_csv("shopping_trends.csv")

print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/iamsouravbanerjee/customer-shopping-trends-dataset?select=shopping_trends.csv

Expected Output:

- A cleaned Pandas data frame ready for analysis.

Import pandas as pd

```
# Import pandas as pd
df = pd.read_csv('shopping_trends.csv')
print('Original DataFrame head:')
display(df.head())
```

Original DataFrame head:

Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type	Discount Applied	Prime Code Used	Previous Purchases	Preferred Payment Method	Frequency of Purchases
8	1	55	Blouse	Clothing	12	Kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express	Yes	Yes	18	Visa	Fortnightly
1	2	18	Sweater	Clothing	64	Mass	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express	Yes	Yes	2	Cash	Fortnightly
2	3	50	Jumps	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Sneakers	Footwear	80	Rockland	M	Maroon	Spring	3.5	Yes	PayPal	Next Day Air	Yes	Yes	40	PayPal	Weekly
4	5	45	Blouse	Clothing	40	Oregon	M	Tan	Spring	2.7	Yes	Cash	Free Shipping	Yes	Yes	31	PayPal	Annually

Handling Missing Values

First, let's check for missing values in the specified columns (age , income , city) and then decide on an appropriate strategy. For numerical columns like age and income , we can fill missing values with the mean or median. For city , we might fill with the mode or a placeholder if appropriate, but given it's a categorical variable to be encoded, dropping rows with missing city might be an option depending on the data context. For now, fill numerical columns with their respective means and check the 'City' column.

```
import pandas as pd
df = pd.read_csv('shopping_trends.csv')

print("Missing values before handling:")
display(df[['Age', 'Purchase Amount (USD)', 'Location']].isnull().sum())

# Fill missing 'Age' and 'Purchase Amount (USD)' with their mean
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Purchase Amount (USD)'] = df['Purchase Amount (USD)'].fillna(df['Purchase Amount (USD)'].mean())

# For 'Location' (city), if there are missing values, we can fill with the mode or drop rows.
# Let's check if there are any missing values in 'Location' and fill with mode if they exist.
if df['Location'].isnull().any():
    mode_city = df['Location'].mode()[0]
    df['Location'] = df['Location'].fillna(mode_city)

print("\nMissing values after handling:")
display(df[['Age', 'Purchase Amount (USD)', 'Location']].isnull().sum())
```

```

Missing values before handling:
#
Age      0
Purchase Amount (USD)  0
Location  0

dtype: int64

Missing values after handling:
#
Age      0
Purchase Amount (USD)  0
Location  0

dtype: int64

```

```

# Encode 'Gender' using map
df['Gender_encoded'] = df['Gender'].map({'Male': 0, 'Female': 1})

# Encode 'Location' using One-hot Encoding
df = pd.get_dummies(df, columns=['location'], prefix='location')

print("DataFrame head after encoding 'Gender' and 'Location'")
display(df.head())

```

— DataFrame head after encoding 'Gender' and 'Location':

Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Size	Color	Season	Review Rating	...	Location_north_dakota	Location_tennessee	Location_texas	Location_utah	Location_wisconsin	Location_virginia
0	1	0	Skateboard	Clothing	53	L	Gray	Winter	3.1		False	False	False	False	False	False
1	2	0	Sneakers	Clothing	64	L	Maroon	Winter	3.1		False	False	False	False	False	False
2	3	0	Jeans	Clothing	73	S	Maroon	Spring	3.1		False	False	False	False	False	False
3	4	1	Sandals	Footwear	90	M	Maroon	Spring	2.5		False	False	False	False	False	False
4	5	0	Skateboard	Clothing	49	M	Tan/peach	Spring	2.7		False	False	False	False	False	False

5 rows x 16 columns

The data has now been cleaned according to your specifications. You can see the new 'gender_encoded' column and the one-hot encoded 'location' columns in the DataFrame.

Task 6 – Sales Data Preprocessing

Task:

Preprocess sales transaction data using AI assistance.

Instructions:

- Convert date columns into datetime format.
- Create new features (month, year, day of week).
- Normalize sales amount column.
- Detect and handle outliers in dataset.

Sample Code:

```

import pandas as pd

data = pd.read_csv("sales_data.csv")

print(data.head())

```

Dataset link:

https://www.kaggle.com/datasets/vinothkannaece/sales-dataset?select=sales_data.csv

Expected Output:

- A transformed dataset with time-based features and normalized values.

```
18)
276
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Load the sales data
sales_df = pd.read_csv('sales_data.csv')
print("Original sales data DataFrame head:")
display(sales_df.head())

print("\nDataFrame info:")
sales_df.info()

Original sales_data DataFrame head:
Product_ID Sales_Date Sales_Rep Region Sales_Amount Quantity_Sold Product_Category Unit_Cost Unit_Price Customer_Type Discount Payment_Method Sales_Channel Region_and_Sales_Rep
0 1000 2023-03-01 Bob North 5000.00 15 Furniture 150.75 267.22 Returning 0.00 Cash Online North-Bob
1 1001 2023-04-21 Bob West 4354.02 17 Furniture 3816.39 4206.44 Returning 0.11 Cash Retail West-Bob
2 1015 2023-04-21 David South 4031.25 30 Food 361.56 371.40 Returning 0.20 Bank Transfer Retail South-David
3 1072 2023-03-24 Bob South 2167.94 38 Clothing 4230.03 4467.75 New 0.02 Credit Card Retail South-Bob
4 1001 2023-03-28 Charlie East 3750.20 13 Electronics 637.37 662.71 New 0.08 Credit Card Online East-Charlie

DataFrame info:
<class 'pandas.core.frame.DataFrame'>
Int64Index: 1000 entries, 0 to 999
Data columns (total 14 columns):
 # Column Non-Null Count Dtype
---
 0 Product_ID 1000 non-null int64
 1 Sales_Date 1000 non-null object
 2 Sales_Rep 1000 non-null object
```

```
1. Convert Date Columns to Datetime Format & Create New Features

Based on the df.info() output, it seems 'Sales_Date' is a good candidate for a date column. I will convert it to datetime objects and then extract 'year', 'month', and 'day of week' as new features.

# Convert 'Sales_Date' to datetime
sales_df['Sales_Date'] = pd.to_datetime(sales_df['Sales_Date'])

# Create new features
sales_df['OrderYear'] = sales_df['Sales_Date'].dt.year
sales_df['OrderMonth'] = sales_df['Sales_Date'].dt.month
sales_df['OrderDayOfWeek'] = sales_df['Sales_Date'].dt.dayofweek + 1 # Monday=0, Sunday=6

print("DataFrame head with new date features:")
display(sales_df.head())

DataFrame head with new date features:
Product_ID Sales_Date Sales_Rep Region Sales_Amount Quantity_Sold Product_Category Unit_Cost Unit_Price Customer_Type Discount Payment_Method Sales_Channel Region_and_Sales_Rep Order
0 1000 2023-03-01 Bob North 5000.00 15 Furniture 150.75 267.22 Returning 0.00 Cash Online North-Bob
1 1001 2023-04-21 Bob West 4354.02 17 Furniture 3816.39 4206.44 Returning 0.11 Cash Retail West-Bob
2 1015 2023-04-21 David South 4031.25 30 Food 361.56 371.40 Returning 0.20 Bank Transfer Retail South-David
3 1072 2023-03-24 Bob South 2167.94 38 Clothing 4230.03 4467.75 New 0.02 Credit Card Retail South-Bob
4 1001 2023-03-28 Charlie East 3750.20 13 Electronics 637.37 662.71 New 0.08 Credit Card Online East-Charlie
```

```
2. Normalize Sales Amount Column

I'll assume 'Sales_Amount' is the column to normalize. I will use Min-Max scaling to transform the values to a range between 0 and 1.

# Initialize MinMaxScaler
scaler = MinMaxScaler()

# Normalize 'Sales_Amount'
sales_df['Sales_Amount_Normalized'] = scaler.fit_transform(sales_df[['Sales_Amount']])

print("DataFrame head with normalized 'Sales_Amount_Normalized':")
display(sales_df.head())

# View head with normalized 'Sales_Amount_Normalized':
Product_ID Sales_Date Sales_Rep Region Sales_Amount Quantity_Sold Product_Category Unit_Cost Unit_Price Customer_Type Discount Payment_Method Sales_Channel Region_and_Sales_Rep OrderYear
0 1000 2023-03-01 Bob North 5000.00 15 Furniture 150.75 267.22 Returning 0.00 Cash Online North-Bob 2023
1 1001 2023-04-21 Bob West 4354.02 17 Furniture 3816.39 4206.44 Returning 0.11 Cash Retail West-Bob 2023
2 1015 2023-04-21 David South 4031.25 30 Food 361.56 371.40 Returning 0.20 Bank Transfer Retail South-David 2023
3 1072 2023-03-24 Bob South 2167.94 38 Clothing 4230.03 4467.75 New 0.02 Credit Card Retail South-Bob 2023
4 1001 2023-03-28 Charlie East 3750.20 13 Electronics 637.37 662.71 New 0.08 Credit Card Online East-Charlie 2023
```


3. Detect and Handle Outliers

I will detect outliers in the 'Sales_Amount' column using the Interquartile Range (IQR) method. Outliers will be capped at the upper and lower bounds to mitigate their impact.

```
# Calculate IQR for 'Sales_Amount'
Q1 = sales_df['Sales_Amount'].quantile(0.25)
Q3 = sales_df['Sales_Amount'].quantile(0.75)
IQR = Q3 - Q1

# Define outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Detect outliers
outliers_count = sales_df[(sales_df['Sales_Amount'] < lower_bound) | (sales_df['Sales_Amount'] > upper_bound)].shape[0]
print(f"Number of outliers detected in 'Sales_Amount': {outliers_count}")

# Handle outliers by capping
sales_df['Sales_Amount_Capped'] = sales_df['Sales_Amount'].clip(lower=lower_bound, upper=upper_bound)

print("\nDataFrame head with 'Sales_Amount_Capped' (outliers handled):")
display(sales_df.head())

# Verify if capping worked by checking descriptive statistics
print("\nDescriptive statistics for original and capped 'Sales_Amount':")
display(sales_df[['Sales_Amount', 'Sales_Amount_Capped']].describe())
```

Number of outliers detected in 'Sales_Amount': 0

DataFrame head with 'Sales_Amount_Capped' (outliers handled):

Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	Order_Prior	Order_Rank	Order_Rep_Ratio	Sales_Amount
0	10/1	2023-09-30	Sub	North	5263.02	16	Furniture	332.75	329.25	Returning	0.05	Cash	Global	North-East	2023	2	0
1	10/3	2023-09-31	Sub	West	4304.02	17	Furniture	3816.35	4209.44	Returning	0.11	Cash	Global	West-East	2023	4	0
2	10/6	2023-09-31	Global	South	8819.23	38	Food	231.56	373.88	Returning	0.20	Bank Transfer	Global	South-East	2023	6	0
3	10/7	2023-09-24	Sub	South	2167.34	39	Clothing	43.88	440.75	New	0.02	Credit Card	Global	South-West	2023	8	3
4	10/1	2023-09-24	Global	East	3758.28	15	Electronics	637.31	662.73	New	0.02	Credit Card	Global	East-Central	2023	9	0

Descriptive statistics for original and capped 'Sales_Amount':

	Sales_Amount	Sales_Amount_Capped
count	1000.000000	1000.000000
mean	5219.262232	6219.262232
std	3866.736126	3866.736126
min	100.120000	100.120000
25%	1000.191660	1000.191660
50%	5219.262232	5219.262232
75%	10017.483881	10017.483881
max	10000.000000	10017.483881

Task 7 – Healthcare Data Preparation

Task:

Clean and preprocess a healthcare dataset using AI scripts.

Instructions:

- Handle missing values in blood_pressure, cholesterol.
- Encode categorical features
- Scale numerical features (min-max or standard scaling).
- Split dataset into training and testing sets.

Dataset link :

<https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk-factors-dataset>

Expected Output:

- A preprocessed dataset ready for machine learning models.


```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, StandardScaler
from sklearn.metrics import ClassificationReport
from sklearn.metrics import classification_report

# Load the healthcare dataset
# Assuming the dataset is saved 'healthcare_data.csv'. Please change it if different.
try:
    df_health = pd.read_csv('healthcare_data.csv')
    print('Successfully loaded healthcare_data.csv')
except FileNotFoundError:
    print('Healthcare dataset not found. Please specify the file or provide the correct path name.')
    # Creating a dummy dataframe for demonstration if file not found
    data = {
        'patient_id': [1, 2, 3, 4, 5],
        'age': [40, 50, 60, 70, 80],
        'gender': ['M', 'F', 'M', 'F', 'M'],
        'blood_pressure': [120, 130, 140, 150, 160],
        'cholesterol': [180, 190, 200, 210, 220],
        'heart_rate': [70, 75, 80, 85, 90],
        'smoking_status': ['No', 'Yes', 'No', 'Yes', 'No'],
        'medication': ['No', 'Yes', 'No', 'Yes', 'No']
    }
    df_health = pd.DataFrame(data)
    print('Created a dummy dataframe for demonstration.')

print('Healthcare dataset head:')
display(df_health.head())

print('Healthcare data info:')
df_health.info()

# Healthcare data can not found. Please specify the file or provide the correct path name.
# Creating a dummy dataframe for demonstration.

Healthcare healthcare head:

```

patient_id	age	gender	blood_pressure	cholesterol	heart_rate	smoking_status	medication
1	40	Female	120.0	180.0	70	No	Diagn
2	50	Female	130.0	190.0	75	No	Diagn
3	60	Female	140.0	200.0	80	No	Diagn
4	70	Female	150.0	210.0	85	No	Diagn
5	80	Male	160.0	220.0	90	No	Diagn

```

Healthcare data info:
Out[10]: pandas.core.frame.DataFrame
Int64Index: 5 entries, 0 to 4
Data columns (total 8 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   patient_id            5 non-null      int64
 1   age                  5 non-null      int64
 2   gender               5 non-null      object
 3   blood_pressure        5 non-null      float64
 4   cholesterol           5 non-null      float64
 5   heart_rate            5 non-null      float64
 6   smoking_status        5 non-null      object
 7   medication            5 non-null      object
dtypes: float64(4), int64(1), object(3)
memory usage: 4.0+ KB

```

1. Handle Missing Values

I will handle missing values in 'blood_pressure' and 'cholesterol' columns by filling them with the mean of their respective columns. This is a common strategy for numerical features.

```

print('Missing values before handling:')
display(df_health[['blood_pressure', 'cholesterol']].isnull().sum())

# Fill missing blood_pressure with its mean
df_health['blood_pressure'] = df_health['blood_pressure'].fillna(df_health['blood_pressure'].mean())

# Fill missing cholesterol with its mean
df_health['cholesterol'] = df_health['cholesterol'].fillna(df_health['cholesterol'].mean())

print('Missing values after handling:')
display(df_health[['blood_pressure', 'cholesterol']].isnull().sum())

```

Missing values before handling:

```

#
blood_pressure    0
cholesterol       1
dtype: int64

```

Missing values after handling:

```

#
blood_pressure    0
cholesterol       0
dtype: int64

```

2. Encode Categorical Features

Next, I'll identify and encode categorical features. For nominal categorical features, One-Hot Encoding is a suitable method. I'll automatically detect object-type columns as categorical.

```
(11) ✓ 06 # Identify categorical features
categorical_features = df_health.select_dtypes(include='object').columns
print(f"Categorical features identified: {list(categorical_features)}")

# Apply One-Hot Encoding
df_health_encoded = pd.get_dummies(df_health, columns=categorical_features, drop_first=True)

print("\nDataFrame head after encoding categorical features:")
display(df_health_encoded.head())
```

Categorical features identified: ['gender', 'smoking_status', 'medication']

DataFrame head after encoding categorical features:

	patient_id	age	blood_pressure	cholesterol	heart_rate	gender_Male	smoking_status_Yes	medication_DrugA	medication_DrugC
0	1	50	129.000000	250.0	75	False	True	True	False
1	2	51	138.000000	165.0	94	False	True	True	False
2	3	32	129.444444	153.0	77	False	True	False	False
3	4	40	95.000000	154.0	93	False	True	True	False
4	5	51	101.000000	223.0	87	True	False	True	False

3. Scale Numerical Features

I will scale the numerical features using `MinMaxScaler` to bring them into a common range (0 to 1). This is important for many machine learning algorithms. I will exclude 'patient_id' from scaling if it exists, as it's typically an identifier and not a feature.

```
(12) ✓ 06 # Identify numerical features for scaling (excluding patient_id and already encoded/hot-encoded columns)
numerical_features = df_health_encoded.select_dtypes(include=[ 'int64', 'float64']).columns.tolist()
if 'patient_id' in numerical_features:
    numerical_features.remove('patient_id')

print(f"Numerical features to scale: {numerical_features}")

# Initialize MinMaxScaler
scaler = MinMaxScaler()

# Apply scaling
df_health_encoded[numerical_features] = scaler.fit_transform(df_health_encoded[numerical_features])

print("\nDataFrame head after scaling numerical features:")
display(df_health_encoded.head())
```

Numerical features to scale: ['age', 'blood_pressure', 'cholesterol', 'heart_rate']

DataFrame head after scaling numerical features:

	patient_id	age	blood_pressure	cholesterol	heart_rate	gender_Male	smoking_status_Yes	medication_DrugA	medication_DrugC
0	1	0.661817	0.443182	0.693548	0.364615	False	True	True	False
1	2	0.525424	0.545455	0.346774	0.871795	False	True	True	False
2	3	0.203390	0.448232	0.250000	0.436897	False	True	False	False
3	4	0.474576	0.056818	0.250005	0.840154	False	True	True	False
4	5	0.525424	0.125000	0.814516	0.692388	True	False	True	False

4. Split Dataset into Training and Testing Sets

Finally, I will split the preprocessed dataset into training and testing sets. You'll need to define your target variable (y) and features (x). For this example, I'll assume `heart_rate` is the target variable, but you should adjust this based on your specific task.

```
(13) ✓ 06 # Define features (X) and target (y)
# Assuming 'heart_rate' is the target variable. Adjust if your target is different.
if 'heart_rate' in df_health_encoded.columns:
    X = df_health_encoded.drop(columns=['heart_rate', 'patient_id'], errors='ignore') & drop patient_id as well
    y = df_health_encoded['heart_rate']
    print(f"Target variable set to 'heart_rate'.")
else:
    print("Could not find 'heart_rate' column. Please define your target variable (y).")
    X = df_health_encoded.drop(columns=['patient_id'], errors='ignore') & drop patient_id if no target
    y = pd.Series(np.zeros(len(X))) & dummy target if not found

# Split the dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

print("\nData preprocessing complete. The dataset is now split into training and testing sets.")
```

Target variable set to 'heart_rate'.

Shape of X_train: (80, 7)
Shape of X_test: (20, 7)
Shape of y_train: (80,)
Shape of y_test: (20,)

Data preprocessing complete. The dataset is now split into training and testing sets.

Task 8 – Employee Salary Data Preprocessing

Task:

Clean and preprocess an employee salary dataset using AI scripts.

Instructions:

- Handle missing values
- Detect and remove salary outliers.
- Standardize department names (lowercase).
- Apply label encoding to job location, department.

Dataset link:

<https://www.kaggle.com/code/vishantmathur/employee-salary>

Expected Output:

- A structured dataset ready for HR analytics.

```
[26]
✓ On import pandas as pd
from sklearn.preprocessing import LabelEncoder
import numpy as np

# Load the employee salary dataset
# Assuming the dataset is named 'employee_salary.csv'. Please change if different.
try:
    df_salary = pd.read_csv('employee_salary.csv')
    print("Successfully loaded employee_salary.csv")
except FileNotFoundError:
    print("employee_salary.csv not found. Creating a dummy DataFrame for demonstration.")
    # Creating a dummy DataFrame for demonstration if file not found
    data = {
        'Employee_ID': range(1, 101),
        'Age': np.random.randint(22, 60, 100),
        'Department': np.random.choice(['Sales', 'Marketing', 'IT', 'HR', 'Finance'], 100),
        'Job_Location': np.random.choice(['New York', 'London', 'Paris', 'Tokyo'], 100),
        'Years_Experience': np.random.randint(0, 30, 100),
        'Salary': np.random.randint(30000, 120000, 100).astype(float) # Ensure salary is float to allow NaN
    }
    # Introduce some missing values and outliers for demonstration
    data['Salary'][np.random.choice(100, 5)] = np.nan # 5 missing salaries
    data['Salary'][np.random.choice(100, 2)] = 500000 # 2 salary outliers
    data['Department'][np.random.choice(100, 3)] = np.nan # 3 missing departments
    data['Job_Location'][np.random.choice(100, 2)] = np.nan # 2 missing job locations
    df_salary = pd.DataFrame(data)

print("\nEmployee Salary DataFrame Head:")
display(df_salary.head())

print("\nDataFrame Info:")
df_salary.info()

print("\nMissing values before handling:")
display(df_salary.isnull().sum())
```

```
employee_salary.csv not found. Creating a dummy DataFrame for demonstration.

Employee Salary DataFrame Head:
Employee_ID  Age  Department  Job_Location  Years_Experience  Salary
0           1    40          Sales          London           17  80507.0
1           2    54           IT            nan             9  31826.0
2           3    31           IT            Tokyo            17  70778.0
3           4    40          Sales          New York           3  90560.0
4           5    39           IT            London            12  30818.0

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 6 columns):
 #   Column              Non-Null Count  Dtype
---  --
 0   Employee_ID         100 non-null   int64
 1   Age                 100 non-null   int64
 2   Department          100 non-null   object
 3   Job_Location        100 non-null   object
 4   Years_Experience    100 non-null   int64
 5   Salary              95 non-null    float64
dtypes: float64(1), int64(3), object(2)
memory usage: 4.8+ KB

Missing values before handling:
#
Employee_ID  0
Age          0
Department   0
Job_Location 0
Years_Experience 0
Salary       4

dtype: int64
```

```
1. Handle Missing Values

I will handle missing values for numerical columns (like 'Salary' and 'Years_Experience') by filling them with the mean. For categorical columns ('Department', 'Job_Location'), I will fill missing values with the mode.

# Fill missing numerical values with the mean
for col in ['Salary', 'Years_Experience']:
    if df_salary[col].isnull().any():
        df_salary[col] = df_salary[col].fillna(df_salary[col].mean())

# Fill missing categorical values with the mode
for col in ['Department', 'Job_Location']:
    if df_salary[col].isnull().any():
        df_salary[col] = df_salary[col].fillna(df_salary[col].mode()[0])

print("Missing values after handling:")
display(df_salary.isnull().sum())

Missing values after handling:
#
Employee_ID  0
Age          0
Department   0
Job_Location 0
Years_Experience 0
Salary       0

dtype: int64
```

2. Detect and Remove Salary Outliers

I will use the Interquartile Range (IQR) method to detect outliers in the 'Salary' column. Rows identified as outliers will be removed from the dataset.

```
(19)
# In
Q1 = df_salary['Salary'].quantile(0.25)
Q3 = df_salary['Salary'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outliers
outliers_before_removal = df_salary[(df_salary['Salary'] < lower_bound) | (df_salary['Salary'] > upper_bound)]
print("Number of salary outliers detected: {}".format(len(outliers_before_removal)))

# Remove outliers
df_salary_cleaned = df_salary[~((df_salary['Salary'] < lower_bound) | (df_salary['Salary'] > upper_bound))].copy()

print("Shape before outlier removal: {}".format(df_salary.shape))
print("Shape after outlier removal: {}".format(df_salary_cleaned.shape))

print("\nDataFrame head after outlier removal:")
display(df_salary_cleaned.head())
```

Number of salary outliers detected: 2

Shape before outlier removal: (106, 6)

Shape after outlier removal: (98, 6)

DataFrame head after outlier removal:

	Employee_ID	Age	Department	Job_Location	Years_Experience	Salary
0	1	40	Sales	London	17	80507.0
1	2	54	IT	nan	9	31826.0
2	3	31	IT	Tokyo	17	70778.0
3	4	40	Sales	New York	3	80580.0
4	5	39	IT	London	12	30918.0

3. Standardize Department Names (lowercase)

I will convert all department names in the 'Department' column to lowercase to ensure consistency.

```
(20)
# In
print("Department names before standardization (sample):")
display(df_salary_cleaned['Department'].value_counts().head())

df_salary_cleaned['Department'] = df_salary_cleaned['Department'].str.lower()

print("\nDepartment names after standardization (sample):")
display(df_salary_cleaned['Department'].value_counts().head())
```

Department names before standardization (sample):

	count
Department	
IT	21
HR	21
Sales	19
Finance	18
Marketing	18

dtype: int64

Department names after standardization (sample):

	count
Department	
it	21
hr	21
sales	19
finance	18
marketing	18

dtype: int64

```
4. Apply Label Encoding to Job Location and Department

Finally, I will apply Label Encoding to the 'Job_Location' and 'Department' columns. This will convert their categorical text values into numerical labels.

In [10]: le = LabelEncoder()
df_salary_cleaned['Job_Location_Encoded'] = le.fit_transform(df_salary_cleaned['Job_Location'])
df_salary_cleaned['Department_Encoded'] = le.fit_transform(df_salary_cleaned['Department'])

print("\nDataFrame head after label encoding:")
display(df_salary_cleaned.head())

print("\nEmployee salary data preprocessing complete.")

***
DataFrame head after label encoding:
   Employee_ID  Age  Department  Job_Location  Years_Experience  Salary  Job_Location_Encoded  Department_Encoded
0            1   40         sales      London             17  80507.0                0                5
1            2   54            IT            nan             9   31826.0                4                2
2            3   31            IT           Tokyo             17   70770.0                3                2
3            4   40         sales     New York             3   90560.0                1                5
4            5   29            IT      London             12  30918.8                0                2

Employee salary data preprocessing complete.
```

Task 9 –Loan Approval Dataset Cleaning

Use AI to generate preprocessing code for a loan dataset.

Instructions:

- Handle missing values in loan_amount, credit_score.
- Encode loan_status (Approved/Rejected).
- Scale numerical features using standard scaling.
- Split the dataset into training and testing sets.

Sample Code:

```
import pandas as pd
data = pd.read_csv("loadapproval.csv")
print(data.head())
```

Dataset link:

<https://www.kaggle.com/datasets/amineipad/loan-approval-dataset>

Expected Output:

- A machine-learning-ready loan dataset.


```
[14]:
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

# Load the dataset
df_loan = pd.read_csv('loanapproval.csv')
print("Original DataFrame Head:")
display(df_loan.head())

print("\nDataFrame Info:")
df_loan.info()

print("\nMissing values before handling:")
display(df_loan.isnull().sum())

Original DataFrame Head:
  applicant_id  age  gender  marital_status  annual_income  loan_amount  credit_score  num_dependents  existing_loans_count  employment_status  loan_approved
0             1   59   Male         Divorced         100073          7169          793             1             1             Unemployed             1
1             2   49   Male         Married         112187         23556          789             0             2             Employed             1
2             3   35   Male         Divorced         34428         27052          372             1             4             Unemployed             0
3             4   63   Female        Single         124195         11313          888             3             4             Self-employed             1
4             5   28   Female         Married         81627         11315          689             0             1             Unemployed             1

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Int64Index: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column              Non-Null Count  Dtype
---  --
 0   applicant_id        1000 non-null   int64
 1   age                 1000 non-null   int64
 2   gender              1000 non-null   object
 3   marital_status      1000 non-null   object
 4   annual_income       1000 non-null   int64
 5   loan_amount         1000 non-null   int64
 6   credit_score        1000 non-null   int64
 7   num_dependents      1000 non-null   int64
 8   existing_loans_count 1000 non-null   int64
 9   employment_status   1000 non-null   object
10   loan_approved       1000 non-null   int64
dtypes: int64(8), object(3)
memory usage: 85.1+ KB
```

```
Missing values after handling:
  applicant_id  0
  age          0
  gender        0
  marital_status  0
  annual_income  0
  loan_amount   0
  credit_score  0
  num_dependents  0
  existing_loans_count  0
  employment_status  0
  loan_approved  0

dtype: object

Double-click (or enter) to edit

There's nothing at position 10.

3. Scale Numerical Features

I will identify all numerical features (excluding the encoded target variable) and apply StandardScaler to them. This helps in standardizing the range of independent variables or features of the data.

# Identify numerical columns for scaling, excluding the encoded target variable (column 10)
numerical_cols = df_loan.select_dtypes(include='number').columns.tolist()

# Drop 'loan_status' if it's in numerical_cols
numerical_cols.remove('loan_status_encoded')

# 'loan_status' is an numerical_col, it's becoming 'loan_status' as an identifier and not a feature for scaling
numerical_cols.remove('loan_status')

print("Numerical columns to scale: ", numerical_cols)

# Scale numerical columns
scaler = StandardScaler()
df_loan[numerical_cols] = scaler.fit_transform(df_loan[numerical_cols])
print("DataFrame Head after scaling numerical features")
display(df_loan.head())

# Print the numerical columns used for scaling
print("\nNumerical columns used for scaling:")

Numerical columns to scale: ['applicant_id', 'age', 'annual_income', 'loan_amount', 'credit_score', 'num_dependents', 'existing_loans_count', 'loan_approved']

DataFrame Head after scaling numerical features:
  applicant_id  age  gender  marital_status  annual_income  loan_amount  credit_score  num_dependents  existing_loans_count  employment_status  loan_approved
0  -1.72123  -1.201345  Male         Divorced         0.402301  -1.696427  1.351625  -0.888042  -0.717090  Unemployed  0.000707
1  -1.72088  -0.614489  Male         Married         0.880362  -0.287325  1.388694  -1.304325  -0.028728  Employed  0.000707
2  -1.72284  -0.980204  Male         Divorced         0.090444  -0.810245  -1.328808  -0.888042  1.388628  Unemployed  -1.000707
3  -1.70807  1.620191  Female        Single         1.120066  1.243364  1.462371  0.721484  1.388628  Self-employed  0.000707
4  -1.70402  -1.101024  Female         Married         0.000216  -1.688882  0.724584  -1.304325  -0.717090  Unemployed  0.000707
```

```
4. Split the Dataset into Training and Testing Sets.

I will split the preprocessed dataset into 80% training and 20% testing sets. loan_status_encoded will be the target variable ( y ), and
the remaining features will be the independent variables ( X ).

In [ ]:
# Define Features (X) and target (y)
X = df_loan.drop('loan_approved', axis=1)
y = df_loan['loan_approved']

# Drop any non-numeric columns from X that might have been added
X = X.select_dtypes(include='number')

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0, stratify=y)

print("X_train shape: ", X_train.shape)
print("X_test shape: ", X_test.shape)
print("y_train shape: ", y_train.shape)
print("y_test shape: ", y_test.shape)

print("\nX_train Head")
display(X_train.head())
print("\ny_train Head")
display(y_train.head())

X_train shape: (666, 7)
X_test shape: (200, 7)
y_train shape: (666,)
y_test shape: (200,)

X_train head:
  applicant_id  age  annual_income  loan_amount  credit_score  num_dependents  existing_loans_count
029  1.170002  1.146170  0.201882  1.303008  -0.190880  0.721484  1.389622
781  0.008017  -1.546229  -1.81207  0.000000  -0.182181  0.016021  -0.717169
122  -1.307090  0.000499  1.678111  -0.000000  0.031281  0.010221  1.589628
8  -1.702907  0.038478  -0.391531  0.072418  -0.579388  -0.000042  -1.445273
758  0.060394  0.514499  -0.496243  0.440882  -0.014138  -0.000002  1.589628

y_train Head:
  loan_approved
029  -1.045133
781  -1.045133
122  0.000710
8  0.000707
758  0.000710

df_type: float64
```

Task 10 – Social Media Profiles

Preprocess a social media engagement dataset using AI assistance.

Instructions:

- Remove duplicate user profiles.
- Handle missing values
- Encode categorical columns

Dataset link:

<https://www.kaggle.com/datasets/medaxone/top-100-social-media-profiles>

Expected Output:

- A cleaned dataset for engagement prediction.

2. Handle Missing Values

I will fill missing numerical values with the mean and missing categorical values with the mode. This is a common strategy to retain as much data as possible while addressing gaps.

```

# Identify numerical and categorical columns for missing value imputation
numerical_cols = df.social.select_dtypes(include=[np.number]).columns
categorical_cols = df.social.select_dtypes(include=[object]).columns

# Fill missing numerical values with the mean
for col in numerical_cols:
    if df.social[col].isnull().any():
        df.social[col] = df.social[col].fillna(df.social[col].mean())

# Fill missing categorical values with the mode
for col in categorical_cols:
    if df.social[col].isnull().any():
        df.social[col] = df.social[col].fillna(df.social[col].mode()[0])

print("Missing values after handling:")
display(df.social.head())

print("DataFrame head after handling missing values:")
display(df.social.head())

```

Missing values after handling:

	user_id	post_id	platform	user_type	likes	comments	shares	engagement_score
0	user_1	post_1	Twitter	Influencer	150.0	10.0	8.0	0.472146
1	user_2	post_2	Instagram	Regular	120.0	15.0	12.0	0.191182
2	user_2	post_3	Twitter	Influencer	300.0	40.0	45.0	0.837588
3	user_4	post_4	Instagram	Regular	220.0	30.0	70.0	0.188776
4	user_3	post_5	Facebook	Influencer	400.0	70.0	11.0	0.327813

DataFrame head after handling missing values:

	user_id	post_id	platform	user_type	likes	comments	shares	engagement_score
0	user_1	post_1	Twitter	Influencer	150.0	10.0	8.0	0.472146
1	user_2	post_2	Instagram	Regular	120.0	15.0	12.0	0.191182
2	user_2	post_3	Twitter	Influencer	300.0	40.0	45.0	0.837588
3	user_4	post_4	Instagram	Regular	220.0	30.0	70.0	0.188776
4	user_3	post_5	Facebook	Influencer	400.0	70.0	11.0	0.327813

3. Encode Categorical Columns

I will encode the categorical columns (platform, user_type) using One-Hot Encoding. This creates new binary columns for each category, which is suitable for many machine learning models. I will also drop the original categorical columns after encoding.

```

# Identify categorical columns for encoding (excluding 'user_id' and 'post_id' if they are identifiers)
categorical_cols_to_encode = ['platform', 'user_type']

# Apply One-Hot Encoding
df_social_encoded = pd.get_dummies(df_social, columns=categorical_cols_to_encode, drop_first=True, dtype=int)

# Drop original non-numeric identifier columns if they are not needed for modeling
# Example: If 'user_id' and 'post_id' are just identifiers and not features
if 'user_id' in df_social_encoded.columns:
    df_social_encoded = df_social_encoded.drop('user_id', axis=1)
if 'post_id' in df_social_encoded.columns:
    df_social_encoded = df_social_encoded.drop('post_id', axis=1)

print("DataFrame head after encoding categorical features:")
display(df_social_encoded.head())

print("DataFrame info after preprocessing:")
df_social_encoded.info()

```

DataFrame head after encoding categorical features:

	likes	comments	shares	engagement_score	platform_instagram	platform_twitter	platform_facebook	user_type_influencer	user_type_regular	user_type_nan
0	150.0	10.0	8.0	0.472146	0	1	0	1	0	0
1	120.0	15.0	12.0	0.191182	1	0	0	0	0	0
2	300.0	40.0	45.0	0.837588	0	1	0	1	0	0
3	220.0	30.0	70.0	0.188776	1	0	0	0	0	0
4	400.0	70.0	11.0	0.327813	0	0	0	1	0	0

DataFrame info after preprocessing:

```

<class 'pandas.core.frame.DataFrame'>
Int64Index: 5 entries, 0 to 4
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   likes                 5 non-null      float64
 1   comments              5 non-null      float64
 2   shares                5 non-null      float64
 3   engagement_score      5 non-null      float64
 4   platform_instagram    5 non-null      int64
 5   platform_twitter      5 non-null      int64
 6   platform_facebook     5 non-null      int64
 7   user_type_influencer  5 non-null      int64
 8   user_type_regular     5 non-null      int64
 9   user_type_nan         5 non-null      int64
dtypes: float64(8), int64(3)
memory usage: 8.2 KB

```

Task 11 – Weather Dataset Feature Engineering

Task:

Use AI to preprocess a weather dataset for forecasting.

Instructions:

- Convert date column into datetime format.

- Handle missing values in temperature, humidity.
- Create new features (season, week_number).
- Normalize rainfall values.

Dataset link:

<https://www.kaggle.com/code/alsamualalhinai/weather-dataset/input>

Expected Output:

- A processed dataset ready for forecasting models.

```

140
df.to
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Create a dummy weather DataFrame for demonstration purposes
data = {
    'date': pd.to_datetime(pd.date_range(start="2023-01-01", periods=100, freq="D")),
    'temperature': np.random.uniform(0, 40, 100),
    'humidity': np.random.uniform(40, 90, 100),
    'rainfall': np.random.uniform(0, 50, 100),
    'wind_speed': np.random.uniform(5, 30, 100)
}

df_weather = pd.DataFrame(data)

# Introduce some missing values for demonstration
df_weather.loc[df_weather.sample(frac=0.05).index, 'temperature'] = np.nan
df_weather.loc[df_weather.sample(frac=0.02).index, 'humidity'] = np.nan

print("Original DataFrame info with missing values:")
df_weather.info()
print("\nFirst 5 rows of the dummy weather data:")
display(df_weather.head())

```

Original DataFrame info with missing values:

```

<class 'pandas.core.frame.DataFrame'>
Int64Index: 100 entries, 0 to 99
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ---
 0   date        100 non-null    datetime64[ns]
 1   temperature  95 non-null     float64
 2   humidity     98 non-null     float64
 3   rainfall     100 non-null    float64
 4   wind_speed   100 non-null    float64
dtypes: datetime64[ns](1), float64(4)
memory usage: 4.0 KB

```

First 5 rows of the dummy weather data:

	date	temperature	humidity	rainfall	wind_speed
0	2023-01-01	19.970887	64.130902	16.231346	18.934126
1	2023-01-02	24.016421	84.810029	41.953131	18.745962
2	2023-01-03	16.210753	83.145986	23.084837	23.903897
3	2023-01-04	16.510746	84.181362	19.606615	23.994739
4	2023-01-05	11.909402	40.875409	40.970886	12.238832

```

140
df.to
# Convert 'date' column to datetime format (if not already)
df_weather['date'] = pd.to_datetime(df_weather['date'])

print("Date column converted to datetime:")
df_weather.info()

```

Date column converted to datetime:

```

<class 'pandas.core.frame.DataFrame'>
Int64Index: 100 entries, 0 to 99
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ---
 0   date        100 non-null    datetime64[ns]
 1   temperature  95 non-null     float64
 2   humidity     98 non-null     float64
 3   rainfall     100 non-null    float64
 4   wind_speed   100 non-null    float64
dtypes: datetime64[ns](1), float64(4)
memory usage: 4.0 KB

```

2. Handle missing values in temperature and humidity

I'll fill missing values in temperature and humidity with their respective means.

```
## Python
# Handle missing values in "temperature" and "humidity" using mean imputation
df_weather["temperature"] = df_weather["temperature"].mean().fillna(df_weather["temperature"].fillna(df_weather["humidity"].mean()).fillna(df_weather["humidity"].mean()))

print("DataFrame info after handling missing values:")
df_weather.info()
print("Original values after imputation:")
print(df_weather["temperature"], df_weather["humidity"])

# DataFrame info after handling missing values
<class 'pandas.core.frame.DataFrame'>
Int64Index: 500 entries, 0 to 499
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype  
---  --
 0   date        500 non-null    object  
 1   temperature  499 non-null    float64  
 2   humidity     499 non-null    float64  
 3   rainfall     500 non-null    float64  
 4   wind_speed   500 non-null    float64  
dtypes: object(1), float64(4)
memory usage: 4.8 KB

Missing values after imputation:
temperature    0
humidity        0

df_weather.info()
#Out[10]: pandas.core.frame.DataFrame
A value is being set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when using df[col].method(value, inplace=True), try using df.method(locs, value, inplace=True) or df[col] = df[col].method(value, inplace=True) instead, to perform the operation inplace on the original object.

df_weather["temperature"] = df_weather["temperature"].mean(), inplace=True
#Out[10]: pandas.core.frame.DataFrame
A value is being set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when using df[col].method(value, inplace=True), try using df.method(locs, value, inplace=True) or df[col] = df[col].method(value, inplace=True) instead, to perform the operation inplace on the original object.

df_weather["humidity"] = df_weather["humidity"].mean(), inplace=True
```

3. Create new features (season, week_number)

I will extract the week_number from the date and create a season column based on the month.

```
## Python
# Create "week_number" feature
df_weather["week_number"] = df_weather["date"].dt.isocalendar().week.astype(int)

# Create "season" feature
def get_season(month):
    if 3 <= month <= 5: # March, April, May
        return "Spring"
    elif 6 <= month <= 8: # June, July, August
        return "Summer"
    elif 9 <= month <= 11: # September, October, November
        return "Autumn"
    else: # December, January, February
        return "Winter"

df_weather["season"] = df_weather["date"].dt.month.apply(get_season)

print("DataFrame with new features 'week_number' and 'season':")
display(df_weather.head())

# DataFrame with new features 'week_number' and 'season'

```

	date	temperature	humidity	rainfall	wind_speed	week_number	season
0	2023-01-01	18.970807	84.136602	16.231346	16.934126	52	Winter
1	2023-01-02	24.016421	84.916029	43.153131	19.745962	1	Winter
2	2023-01-03	16.216753	83.145988	23.684837	21.903097	1	Winter
3	2023-01-04	16.510746	94.181382	19.109515	23.984730	1	Winter
4	2023-01-05	13.909402	40.871409	40.978800	13.238602	1	Winter

4. Normalize rainfall values

I will normalize the rainfall column using MinMaxScaler to scale values between 0 and 1.

```
## Python
# Normalize "rainfall" values using MinMaxScaler
scaler = MinMaxScaler()
df_weather["rainfall_normalized"] = scaler.fit_transform(df_weather[["rainfall"]])

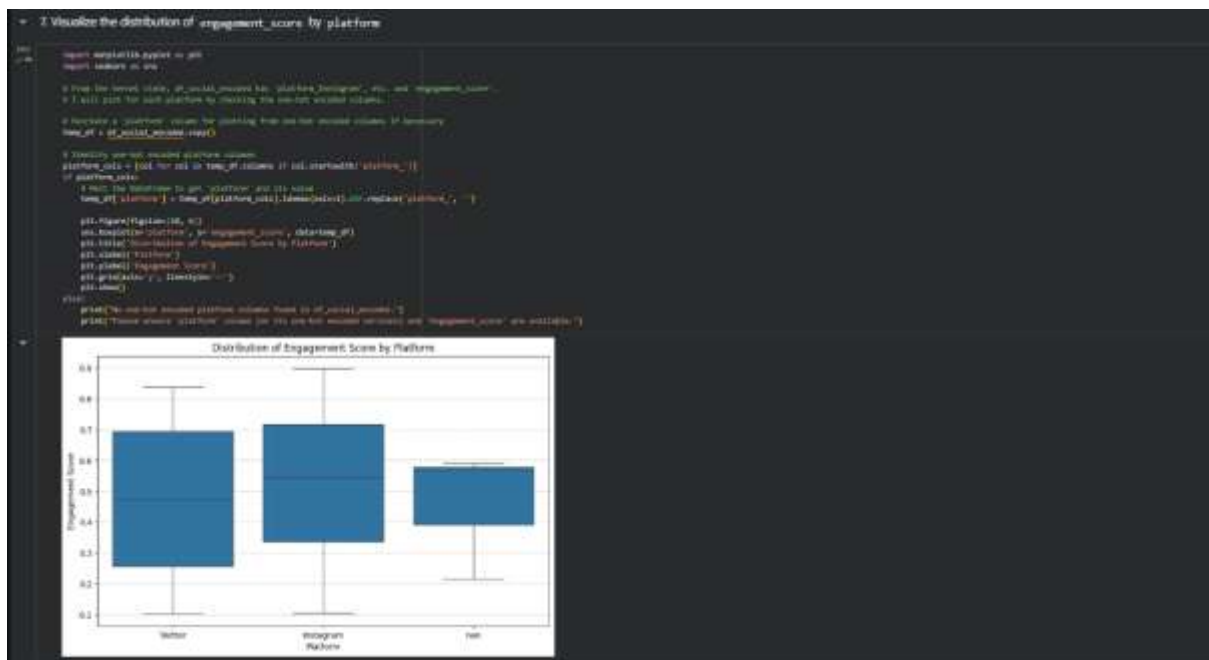
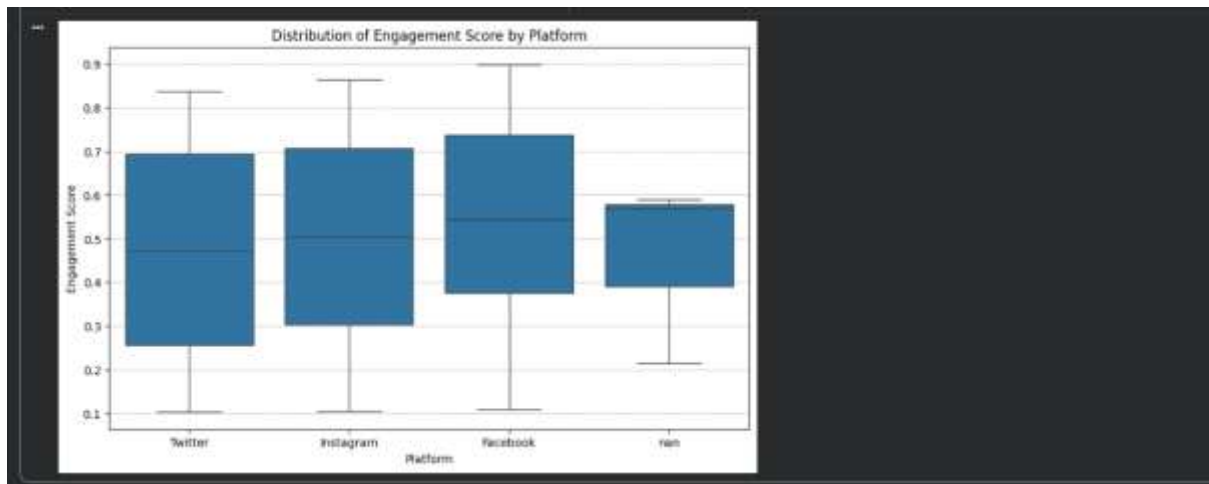
print("DataFrame with normalized 'rainfall' values:")
display(df_weather.head())

# DataFrame with normalized 'rainfall' values:

```

	date	temperature	humidity	rainfall	wind_speed	week_number	season	rainfall_normalized
0	2023-01-01	18.970807	84.136602	16.231346	16.934126	52	Winter	0.314437
1	2023-01-02	24.016421	84.916029	43.153131	19.745962	1	Winter	0.809543
2	2023-01-03	16.216753	83.145988	23.684837	21.903097	1	Winter	0.498122
3	2023-01-04	16.510746	94.181382	19.109515	23.984730	1	Winter	0.373782
4	2023-01-05	13.909402	40.871409	40.978800	13.238602	1	Winter	0.824546

The weather dataset has now been preprocessed according to your instructions. You can replace the dummy DataFrame with your actual data by modifying the first code cell to load your .csv file.



Task 12 – Movie Ratings Dataset Preparation

Task:

Clean and preprocess a movie ratings dataset using AI-generated scripts.

Instructions:

- Handle missing values in rating, genre.
- Encode categorical variables like genre, language.
- Remove duplicate movie entries.
- Scale rating values between 0 and 1.

Dataset link:

<https://www.kaggle.com/datasets/PromptCloudHQ/imdb-data>

Expected Output:

- A dataset suitable for recommendation systems.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.metrics import mean_squared_error

# Create a dummy movie ratings DataFrame
data = {
    'movie_id': range(1, 11), # Range to 11 to create 10 movies
    'title': ['Movie 1'] * 2, ['Movie 2'] * 2, ['Movie 3'] * 2, ['Movie 4'] * 2, ['Movie 5'] * 2,
    'genre': ['Action', 'Comedy', 'Drama', 'Fantasy', 'Horror', 'Mystery', 'Romance', 'Sci-Fi', 'Thriller', 'Western'] * 2,
    'language': ['English', 'Spanish', 'English', 'French', 'English', 'Spanish', 'English', 'French', 'English', 'Spanish'] * 2,
    'rating': np.random.randn(10, 100).ravel(),
    'release_year': np.random.randint(1980, 2020, 100)
}

df_movies = pd.DataFrame(data)

# Introduce some missing values for demonstration
df_movies.loc[df_movies.sample(n=10).index, 'rating'] = np.nan
df_movies.loc[df_movies.sample(n=10).index, 'genre'] = np.nan

print("Original DataFrame info with missing values and potential duplicates:")
df_movies.info()
print("\nFirst 5 rows of the dummy movie ratings data:")
df_movies.head()
```

Original DataFrame info with missing values and potential duplicates:

```
Out[1]: pandas.core.frame.DataFrame
Int64Index: 110 entries, 0 to 109
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  ---
 0   movie_id    110 non-null      int64
 1   title       110 non-null      object
 2   genre       100 non-null      object
 3   language    110 non-null      object
 4   rating      90 non-null       float64
 5   release_year 110 non-null      int64
dtypes: float64(1), int64(3), object(2)
memory usage: 7.3+ MB
```

First 5 rows of the dummy movie ratings data:

movie_id	title	genre	language	rating	release_year
0	Movie 1	Action	English	0.218034	1980
1	Movie 2	Comedy	Spanish	nan	2000
2	Movie 3	Drama	English	0.897127	2000
3	Movie 4	Sci-Fi	French	0.737780	2004
4	Movie 5	Action	English	0.440206	1980

4. Handle missing values in rating and genre

Fill the missing 'rating' values with the mean and missing 'genre' values with the mode.

```
# Handle missing values in 'rating' with the mean
df_movies['rating'].fillna(df_movies['rating'].mean(), inplace=True)

# Handle missing values in 'genre' with the mode
mode_genre = df_movies['genre'].mode()[0]
df_movies['genre'].fillna(mode_genre, inplace=True)

print("DataFrame info after handling missing values:")
df_movies.info()
print("\nOutliers values after imputation:")
print(df_movies[['rating', 'genre']].isnull().sum())

# DataFrame info after handling missing values:
Out[1]: pandas.core.frame.DataFrame
Int64Index: 110 entries, 0 to 109
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  ---
 0   movie_id    110 non-null      int64
 1   title       110 non-null      object
 2   genre       110 non-null      object
 3   language    110 non-null      object
 4   rating      110 non-null      float64
 5   release_year 110 non-null      int64
dtypes: float64(1), int64(3), object(2)
memory usage: 7.3+ MB
```

Missing values after imputation:

```
rating      0
genre       0
dtypes: int64
Out[1]: int64
```

Warning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 0.20. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing `df[col].method(value, inplace=True)`, try using `df.method(iol: value), inplace=True` or `df[iol] = df[iol].method(value)` instead, to perform the operation inplace on the original object.

```
df_movies['rating'].fillna(df_movies['rating'].mean(), inplace=True)
df_movies['genre'].fillna(mode_genre, inplace=True)
```

Warning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 0.20. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing `df[col].method(value, inplace=True)`, try using `df.method(iol: value), inplace=True` or `df[iol] = df[iol].method(value)` instead, to perform the operation inplace on the original object.

```
df_movies['genre'].fillna(mode_genre, inplace=True)
```

2. Remove duplicate movie entries

I will remove duplicate entries based on 'title' and 'release_year' to ensure unique movie records.

```
[10]: # Get initial number of rows
initial_rows = df_movies.shape[0]

# Remove duplicate movie entries based on 'title' and 'release_year'
df_movies.drop_duplicates(subset=['title', 'release_year'], inplace=True)

# Get number of rows after removing duplicates
rows_after_duplicates = df_movies.shape[0]

print(f"Initial number of rows: {initial_rows}")
print(f"Number of rows after removing duplicates: {rows_after_duplicates}")
print(f"Number of duplicate entries removed: {initial_rows - rows_after_duplicates}")
display(df_movies.head())
```

Initial number of rows: 118
Number of rows after removing duplicates: 110
Number of duplicate entries removed: 8

	movie_id	title	genre	language	rating	release_year
0	1	Movie 1	Action	English	8.310124	1996
1	2	Movie 2	Comedy	Spanish	5.527336	2000
2	3	Movie 3	Drama	English	9.107127	2009
3	4	Movie 4	Sci-Fi	French	3.730768	2004
4	5	Movie 5	Action	English	9.441030	1996

3. Encode categorical variables like 'genre' and 'language'

I will use One-Hot Encoding for 'genre' and 'language' columns.

```
[11]: # One-hot encode 'genre' and 'language'
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

# Fit and transform the categorical columns
encoded_features = encoder.fit_transform(df_movies[['genre', 'language']])

# Create a DataFrame from the encoded features
encoded_df = pd.DataFrame(encoded_features.get_feature_names_out(['genre', 'language']), index=df_movies.index)

# Concatenate the encoded DataFrame with the original DataFrame and drop original categorical columns
df_movies_encoded = pd.concat([df_movies.drop(columns=['genre', 'language']), encoded_df], axis=1)

print("DataFrame after One-Hot Encoding categorical variables:")
display(df_movies_encoded.head())
```

DataFrame after One-Hot Encoding categorical variables:

	movie_id	title	rating	release_year	genre_Action	genre_Comedy	genre_Drama	genre_Sci-Fi	genre_Thriller	language_english	language_french	language_spanish
0	1	Movie 1	8.310124	1996	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
1	2	Movie 2	5.527336	2000	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
2	3	Movie 3	9.107127	2009	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0
3	4	Movie 4	3.730768	2004	0.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0
4	5	Movie 5	9.441030	1996	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0

4. Scale rating values between 0 and 1

I will use 'MinMaxScaler' to scale the 'rating' column.

```
[12]: # Scale 'rating' values between 0 and 1 using MinMaxScaler
scaler = MinMaxScaler()
df_movies_encoded['rating_scaled'] = scaler.fit_transform(df_movies_encoded[['rating']])

print("DataFrame with scaled 'rating' values:")
display(df_movies_encoded.head())
```

DataFrame with scaled 'rating' values:

	movie_id	title	rating	release_year	genre_Action	genre_Comedy	genre_Drama	genre_Sci-Fi	genre_Thriller	language_english	language_french	language_spanish	rating_scaled
0	1	Movie 1	8.310124	1996	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.627621
1	2	Movie 2	5.527336	2000	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.516420
2	3	Movie 3	9.107127	2009	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	0.910740
3	4	Movie 4	3.730768	2004	0.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.385412
4	5	Movie 5	9.441030	1996	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.956626